МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

Факультет електроніки і комп’ютерних технологій

Звіт

про виконання лабораторної роботи №14

з курсу «Вибрані розділи науки про дані»

«Розгортання та моніторинг дрейфу»

Виконав:

студент групи ФЕІм-13с

Шевчук Максим Вікторович

Перевірив:

Парубочий В. О.

Львів 2026

Мета та початкові матеріали

Мета роботи - підготувати систему ЛР12 до контейнерного запуску, додати експлуатаційні метрики, перевірити виявлення дрейфу та описати реагування й безпечне розгортання.

Використано наявну Energy GraphRAG: FastAPI, Neo4j, FAISS, multilingual MiniLM і локальну FLAN-T5-small. Новий запуск виконано в Docker на диску D. Система доступна через локальний порт 8144; зовнішній LLM API не потрібен.

Таблиця 1 - Походження матеріалів

Джерело	Повторно використано	Нове в ЛР14
ЛР12	API, Engine, імпортер, індекс, вебформа	Docker API, метрики, розширені журнали
ЛР11	Ваги моделей і тестові питання	50 HTTP-запитів до контейнера
ЛР8–9, ЛР1	Граф, вимірювання, прогнози	Reference та модельовані зсуви

П’ять результатів: контейнеризація; журнали й метрики; PSI/KS та графіки; тривоги й контрольні сценарії; runbook, план розгортання та контракт даних. Успадковані матеріали не подано як нові вимірювання.

Середовище: Docker Engine 28.2.2, Linux/WSL2, Python 3.11.18, CPU, seed=42. Основні версії: NumPy 1.26.4, SciPy 1.15.3, FastAPI 0.115.6, PyTorch 2.6.0+cpu. Повний pip freeze збережено в requirements-lock.txt.

1 Контейнеризація системи

Compose містить три сервіси: neo4j, одноразовий init та api. Neo4j зберігає граф у власному іменованому томі. Init чекає healthy-бази й виконує MERGE; API запускається після успішного завершення init. Це усуває ручний імпорт після docker compose up.

Базовий образ API - python:3.11-slim. Застосунок працює від UID/GID 10001, EXPOSE 8144, CMD запускає один uvicorn worker. Для LLM використано CPU-версію PyTorch. Dockerfile не копіює .env, моделі, журнали чи звіти.

Пароль випадково створює setup_env.py; він зберігається у локальному .env, виключеному з образу та ZIP. Neo4j не публікує порти на хості. Ваги монтуються read-only з D:/LNU/DS/11/models; дані та результати роботи знаходяться в D:/LNU/DS/14.

```
python setup_env.py --models-dir D:/LNU/DS/11/models
docker compose -p lnu-ds14 up -d --build
docker compose -p lnu-ds14 ps
curl.exe http://127.0.0.1:8144/health
```

Підготовка .env і ваг виконується один раз. Після цього система стартує однією командою Compose. Healthcheck API виконує /health, який також перевіряє Neo4j; health не є повним тестом якості генерації.

Фактичний запуск підтвердив healthy для API та Neo4j. Імпортовано 43 746 вузлів і 199 287 зв’язків; FAISS містить 38 фрагментів. Image IDs, версія Engine та контрольні суми джерел збережені в results/container_evidence.json.

1 Визначення образу API

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 \
    HF_HOME=/tmp/huggingface TRANSFORMERS_CACHE=/tmp/huggingface/hub \
    HF_HUB_OFFLINE=1 TOKENIZERS_PARALLELISM=false MODELS_DIR=/models
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --index-url
https://download.pytorch.org/whl/cpu torch==2.6.0 \
    && pip install --no-cache-dir -r requirements.txt \
    && groupadd --gid 10001 app && useradd --uid 10001 --gid app
--create-home app
COPY --chown=10001:10001 . /app
USER 10001:10001
EXPOSE 8144
HEALTHCHECK --interval=15s --timeout=5s --start-period=120s --retries=12 \
    CMD python -c "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8144/health',timeout=4)"
CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8144",
"--workers", "1", "--no-access-log"]
```

Прямі залежності зафіксовані у requirements.txt; фактичні транзитивні версії - у requirements-lock.txt. У маніфесті зафіксовано revisions моделей та версію промпту. Тег образу придатний для локальної демонстрації; для виробничого розгортання план вимагає незмінний digest.

2 Журнали та експлуатаційні метрики

Для кожного прикладного HTTP-запиту створюється JSON-запис: UTC-час, UUID запиту, шаблон маршруту, статус, версії моделі та промпту, затримка, токени й API-вартість. Тексти запитань, відповідей та секрети до журналу не потрапляють.

Ендпоінт /metrics повертає Prometheus-лічильники запитів, помилок, input/output токенів і зовнішньої API-вартості, а також гістограму часу. /health і /metrics виключені з бізнес-лічильників, тому опитування стану не змінює статистику навантаження.

Таблиця 2 - Фактичний прогін 50 запитів

Показник	Значення
HTTP 200 / навмисні HTTP 422	45 / 5
Неочікувані відповіді	0
Токени input / output	14111 / 353
Середній час усіх запитів	1.089 с
p95 усіх / успішних	1.290 / 1.294 с
Перший запит і максимум	28.591 с
Зовнішня API-вартість	$0

Запити виконувалися послідовно всередині API-контейнера. Ваги вже завантажені, але перша генерація окремо не прогрівалася; її 28.591 с включено в результати. p95 та максимум наведено окремо. Це не тест паралельного навантаження.

Усі 50 відповідей зіставлено за request_id з JSON-журналом; прирости лічильників, гістограми та токенів збігаються. Часткові токени невдалої генерації невідомі. $0 означає лише відсутність платного LLM API, без оцінки електроенергії та обладнання.

3 Еталон і методика виявлення дрейфу

Моніторинг охоплює activePowerKw та voltageV вузлів Lab8Observation і forecastKwh вузлів Lab8Forecast. Це поля наявного графа ЛР12, що походять з енергетичних даних ЛР1. Розподіл RAG-запитів або концептуальний дрейф LLM цим експериментом не перевіряється.

Для кожної популяції взято перші 6000 історичних записів, відсортованих за часом. Фіксована випадкова перестановка seed=42 ділить їх на 3000 reference та 3000 current pool без спільних рядків усередині ознаки. Дати й SHA-256 джерела записано в reference_manifest.json.

Таблиця 3 - Популяції для моніторингу

Поле	Історичні межі	Reference / pool
activePowerKw	2006-12-16 2007-08-27	3000 / 3000
voltageV	2006-12-16 2007-08-27	3000 / 3000
forecastKwh	2009-09-09 2010-06-08	3000 / 3000

Ознаки мають різні історичні періоди й не видаються за одну спільну таблицю спостережень. Потужність і прогноз пов’язані, тому підтвердження двома ознаками не є двома незалежними статистичними доказами.

PSI обчислюється як сума (a - e)·ln(a/e) за частками поточного та еталонного розподілів. Десять квантильних інтервалів визначає лише reference; хвости відкриті до ±∞. Псевдолічильник 0,5 усуває log(0), після чого частки знову нормуються.

KS-тест scipy.stats.ks_2samp порівнює емпіричні розподіли. Для трьох ознак використано поріг $p<0{,}01/3$. Часова залежність і повтори значень обмежують буквальну інтерпретацію p-value як частоти помилки.

3 Результати PSI та KS

Таблиця 4 - Порівняння за 3000 значень у кожній вибірці

Сценарій	Поле	PSI	KS D	p-value
Без зсуву	activePowerKw	0.013	0.021	0.52
Без зсуву	voltageV	0.010	0.020	0.56
Без зсуву	forecastKwh	0.009	0.027	0.22
Зсув +1σ	activePowerKw	4.515	0.601	0*
Зсув +1σ	voltageV	1.163	0.402	2.2e-217
Зсув +1σ	forecastKwh	2.975	0.438	2.1e-259
Одиниці ×100	activePowerKw	7.817	1.000	0*
Одиниці ×100	voltageV	7.817	1.000	0*
Одиниці ×100	forecastKwh	7.817	1.000	0*

У сценарії +1σ до current додається стандартне відхилення reference відповідної ознаки. В експерименті без зсуву PSI нижче 0,1 для всіх трьох полів. Навмисний зсув переводить усі поля вище 0,25; множення на 100 дає значно сильнішу відмінність.

Позначка 0* означає числовий нуль, повернутий SciPy для надзвичайно малого p-value; це не твердження про математично нульову ймовірність. Сирі значення збережено у results/drift.json.

Пороги PSI 0,1/0,25 обрано як початкові рівні warning/critical із завдання. Вони перевірені на спокійних вікнах, але не є універсальним стандартом. Тривога дрейфу вимагає одночасно достатнього PSI, малого p-value та щонайменше двох ознак.

3 Розподіли після навмисних змін

На рис. 1 порівняно reference і current після додавання 1σ. Для потужності й прогнозу зсув прибирає масу біля початкового мінімуму, що пояснює великі значення PSI.

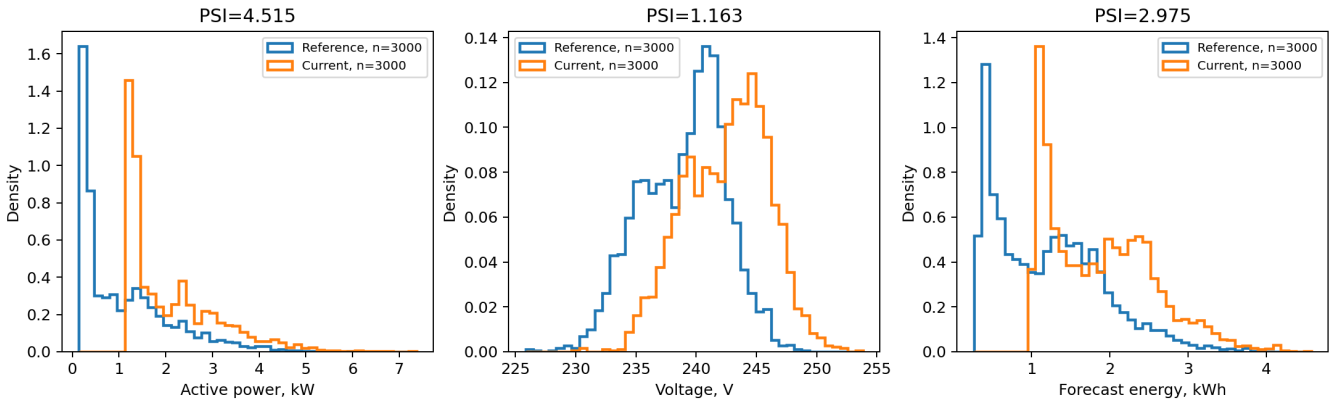


Рис. 1 - Змодельований зсув середнього на 1σ

На рис. 2 усі поточні значення помножено на 100. Розподіли майже не перекриваються; контракт фізичних одиниць також порушено. Такий випадок слід виправляти в джерелі, а не компенсувати перенавчанням.

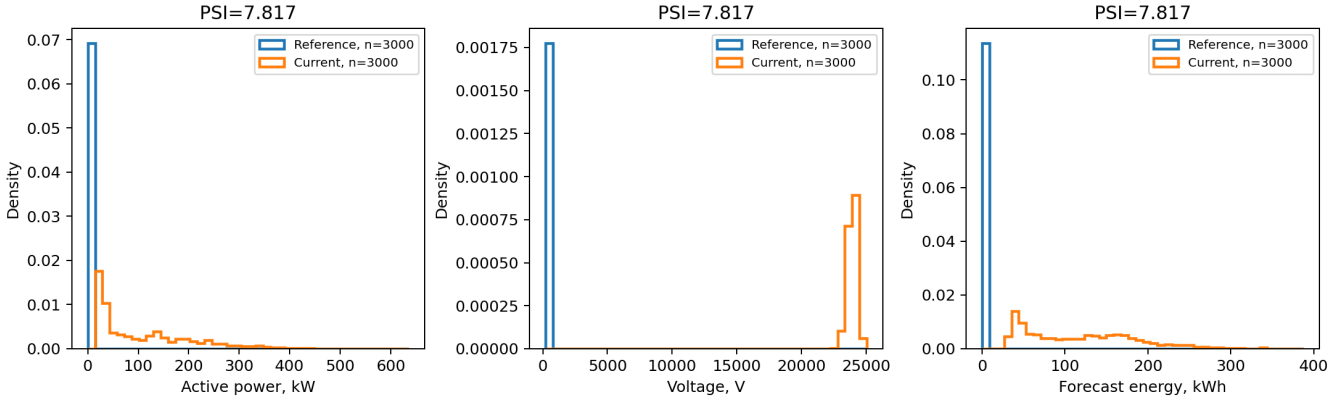


Рис. 2 - Змодельований дефект одиниць ×100

3 Контрольний розподіл і динаміка PSI

На рис. 3 показано початкові незалежні за рядками вибірки без навмисної зміни. Подібність розподілів узгоджується з малим PSI та відсутністю тривоги.

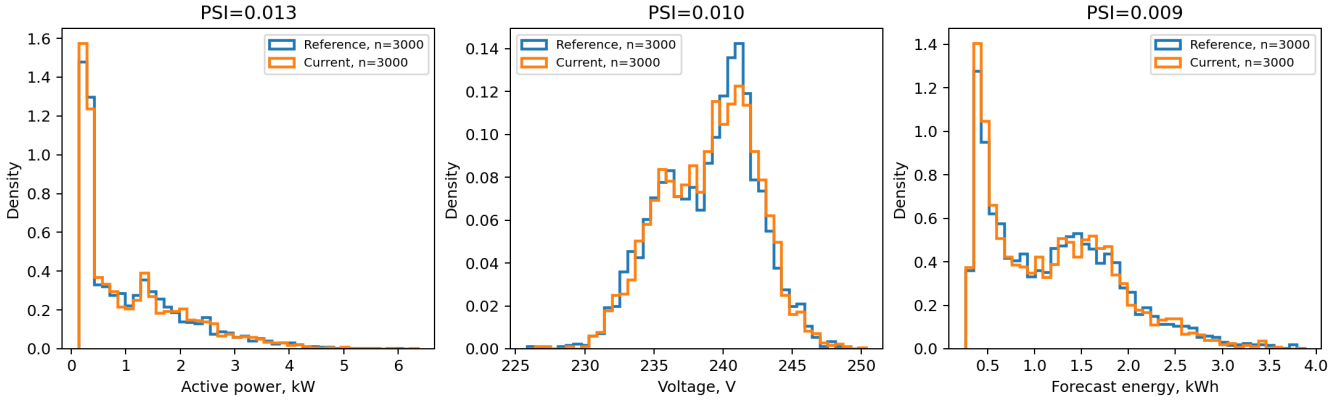


Рис. 3 - Історичний контрольний сценарій без зсуву

На рис. 4 наведено 10 змодельованих тижневих вікон по 1000 значень на ознаку. Послідовність зсувів у σ: 0; 0; 0,1; 0,2; 0,35; 0,5; 0,75; 1; 1; 0. Це демонстрація розвитку й зникнення зсуву, а не 10 тижнів реальної експлуатації.

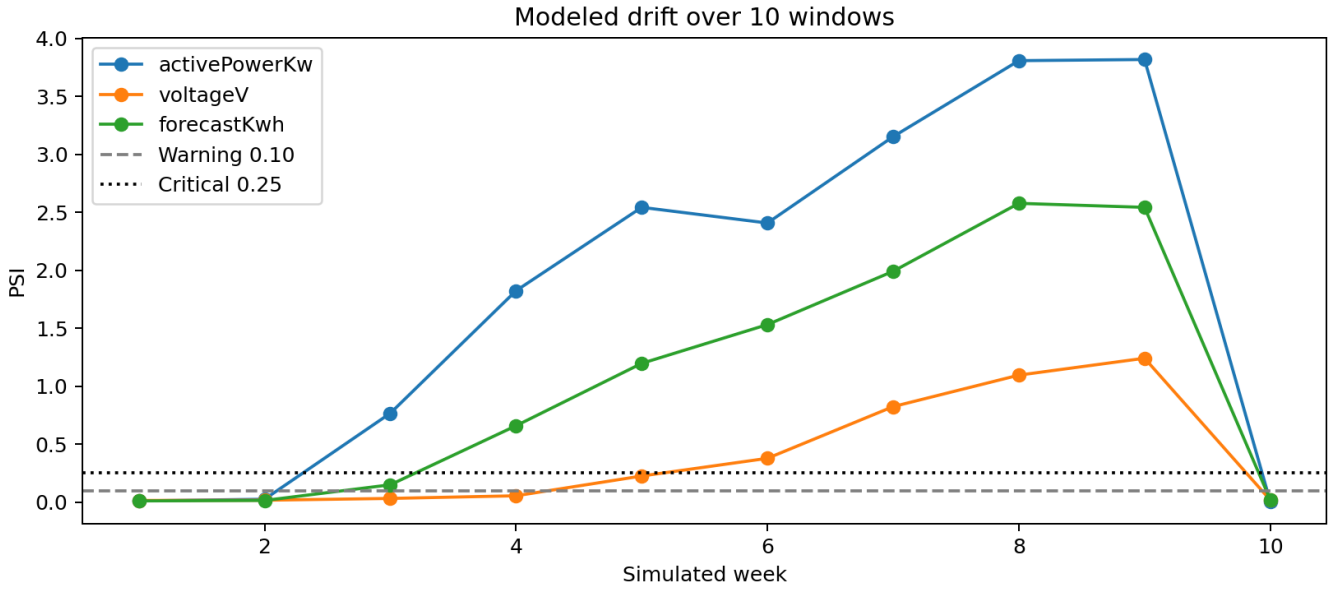


Рис. 4 - PSI у десяти змодельованих тижнях

4 Тривоги та контрольні інциденти

AlertManager створює warning або critical і зберігає час останньої тривоги кожного типу. Повтор протягом 900 с пригнічується; warning→critical проходить негайно. Дефект контракту й відмова LLM не чекають підтвердження двома PSI.

Таблиця 5 - Три змодельовані інциденти

Сценарій	Реакція	Підтвердження
Дрейф +1σ	data_drift, critical	PSI/KS у трьох полях
Дефект ×100	data_defect, critical	Порушення діапазонів контракту
Відмова LLM	llm_failure, critical	HTTP 503 при ін’єкції відмови

Відмова LLM перевірена в окремому процесі: справжні модель і граф завантажено, але метод llm.generate навмисно замінено функцією, що піднімає виняток. API повернуло HTTP 503, а AlertManager - critical. Це контрольована ін’єкція, не спонтанний збій; робочий API не перемикався в режим відмови.

Для дрейфу тривога в момент 0 пройшла, повтор через 60 с пригнічено, після 901 с тривогу дозволено. Тести окремо перевірили warning, ескалацію до critical та незалежні типи інцидентів. У модельованій динаміці третій тиждень має warning, четвертий - critical.

На 100 спокійних bootstrap-вікнах по 1000 значень на поле отримано 0 тривог, тобто 0.0%. Вікна повторно використовують held-out pool і не є незалежними календарними тижнями. Нуль у цій перевірці не доводить нульову частоту хибних тривог у майбутньому.

Зменшення хибних тривог: сезонні еталони, достатній розмір пакета, перевірка двох послідовних вікон, поріг ефекту разом із p-value та окрема валідація одиниць. Вісім автоматичних тестів пройшли; залежності перевірено pip check.

5 Runbook та відповідальність

У навчальній системі ролі оператора, розробника й відповідального за дані виконує Шевчук Максим. Запропонований runbook задає час початку реакції, а не обіцяний час повного відновлення.

Таблиця 6 - Перші дії при інциденті

Тривога	Дія та відповідальний	Реакція
Warning дрейфу	Оператор: перевірити період, сезонність, походження; оцінити наступне вікно	1 година
Critical дрейфу	Дані / розробник: зупинити просування релізу; перевірити джерело; за зв’язку з релізом - відкат	15 хв
Дефект контракту	Дані: карантин пакета, останній перевірений граф; виправити одиниці в джерелі	5 хв
LLM недоступна	Розробник: перевірити журнал, ресурси, завантаження ваг; відновити попередній образ	5 хв

Спочатку слід класифікувати інцидент: дефект схеми чи одиниць, справжня зміна розподілу або недоступність сервісу. Великий PSI сам по собі не доводить погіршення точності моделі.

Перенавчання можливе лише після перевірки коректності джерела, наявності істинних міток та нового offline-оцінювання. Оновлення reference має окрему версію, період і hash; воно не повинно приховувати інцидент. Закриття інциденту, причина й час відновлення фіксуються оператором.

5 План розгортання та контракт даних

Обрано blue-green: green завчасно прогрівається з окремим томом графа; blue зберігається для швидкого повернення. Для невеликої CPU-системи canary має мало запитів для статистики, а shadow подвоює обчислення. У лабораторній запущено одну версію; blue-green тут описаний план, без реалізованого reverse proxy.

Перед перемиканням перевіряються health, API-контракт, точність цитат, тести, 50 запитів і контракт даних. Критерії успіху: відсутність неочікуваних 5xx і дефектів цитат; p95 успішних відповідей не більше 1,2×baseline; відсутність нової critical-тривоги.

Baseline p95 успішних відповідей становить 1.294 с. Порівняння допустиме лише за того самого набору запитів, CPU та рівня прогріву. Перший холодний виклик включено до поточного baseline.

Відкат: три невдалі healthcheck поспіль, дефект цитування або контракту, 5xx>5% за 5 хв при ≥20 запитах чи p95>1,5×baseline у двох вікнах. Оператор повертає трафік на blue та зберігає журнал green. Це запропоновані початкові SLO, не production-гарантії.

Data contract 1.0 задає унікальний id, ISO localdatetime без вигаданого UTC offset, скінченні числові значення та одиниці: activePowerKw у kW, voltageV у V, forecastKwh у kWh. Навчальні діапазони - 0..20, 180..270 і 0..20 відповідно; вони не є універсальними нормами електромережі.

Для дрейфу потрібні ≥1000 значень на поле; пропуски не замінюються нулями. Новий пакет має version, period_start/end, units, record_count і SHA-256. Порушений пакет відхиляється, останній перевірений граф зберігається. Докладні дії й запропонований SLA записано в RUNBOOK.md, DEPLOYMENT.md та DATA_CONTRACT.md.

Висновки та підготовка до захисту

Систему ЛР12 запущено в Docker із власним томом Neo4j, автоматичним імпортом та непривілейованим API. За 50 HTTP-запитів отримано 45 успішних відповідей і 5 очікуваних помилок валідації; неочікуваних відповідей не було. Метрики й токени звірено з журналами.

PSI та KS виявили обидва навмисні зсуви трьох полів. Контракт відокремив ×100 як дефект одиниць; контрольована відмова генерації дала HTTP 503 і critical-тривогу. На 100 спокійних bootstrap-вікнах тривог не було. Межі експерименту - історичні дані, корельовані поля й модельовані тижні.

1. Дрейф даних - зміна P(X); дрейф концепції - зміна залежності P(Y|X). Поточний дослід перевіряє перше, без доведення зміни якості за істинними мітками.

2. PSI=0,3 перевищує початковий critical-поріг 0,25, але потрібні перевірка вибірки, одиниць та умов; це не відсоток втрати точності.

3. Shadow обробляє копію трафіку без впливу на відповідь; canary отримує малу частину живого трафіку; blue-green перемикає між двома готовими середовищами.

4. Реєстр моделей зберігає версії, походження, метрики й статуси просування та допомагає відкату. У цій роботі використано файловий маніфест; сервер MLflow не розгортався.

5. Дефект одиниць змінює числовий розподіл і тому виглядає як дрейф. Data contract допомагає визначити причину до рішення про перенавчання.

Джерела та використання ШІ

1. Завдання ЛР14 «Розгортання та моніторинг дрейфу», курс «Вибрані розділи науки про дані», 2026–2027 н. р. Копія: sources/assignment14.pdf.

2. Власні матеріали ЛР1, ЛР8–9, ЛР11 та ЛР12: енергетичні дані, граф знань, корпус, ваги й код API. SHA-256 повторно використаних файлів: data/provenance.json.

3. Docker. Control startup and shutdown order in Compose. https://docs.docker.com/compose/how-tos/startup-order/ .

4. SciPy. scipy.stats.ks_2samp. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html .

5. Prometheus Python client. Histogram. https://prometheus.github.io/client_python/instrumenting/histogram/ .

6. Hebrail G., Berard A. Individual Household Electric Power Consumption. UCI Machine Learning Repository. DOI: 10.24432/C58K54. CC BY 4.0. Початковий набір описує одне домогосподарство у Франції, не енергосистему України.

Для аналізу завдання, написання коду, документації та оформлення використано OpenAI Codex. Користувач доручив переглянути попередні роботи, підготував Docker та попросив зберігати матеріали на D у папці з номером лабораторної. Повні фактичні формулювання й призначення ШІ наведені в AI_DECLARATION.md.

Перевірка допомоги ШІ: реальні Docker-запуски, 50 HTTP-запитів, 8 тестів, pip check, зіставлення метрик із журналами, PSI/KS на збережених даних, ін’єкція відмови та візуальна перевірка графіків і PDF. Контрольовані сценарії не видаються за виробничі інциденти. Автор має самостійно розуміти код перед захистом.

Додаток А PSI та правила тривог

Лістинг програми drift.py. Оригінальний виконуваний файл включено до ZIP. Успадкований Engine ЛР12 повторно не друкується; тести та службові скрипти не включено до лістингу звіту.

```
import numpy as np
from scipy.stats import ks_2samp


def finite_sample(values):
    x = np.asarray(values, dtype=float)
    if x.ndim != 1 or len(x) < 2 or not np.isfinite(x).all():
        raise ValueError('Expected at least two finite numeric values')
    return x


def psi(expected, actual, bins=10):
    expected, actual = finite_sample(expected), finite_sample(actual)
    if bins < 2:
        raise ValueError('At least two bins required')
    # Freeze edges on reference only; open tails count out-of-range
observations.
    inner = np.unique(np.quantile(expected, np.arange(1, bins) / bins))
    if np.ptp(expected) == 0:
        center = expected[0]
        tolerance = np.sqrt(np.finfo(float).eps) * max(1, abs(center))
        edges = np.array([-np.inf, center-tolerance, center+tolerance,
np.inf])
    else:
        edges = np.r_[-np.inf, inner, np.inf]
    e = np.histogram(expected, edges)[0].astype(float)
    a = np.histogram(actual, edges)[0].astype(float)
    # Jeffreys pseudocount prevents log(0); both vectors remain
normalized.
    e = (e + .5) / (e.sum() + .5 * len(e))
    a = (a + .5) / (a.sum() + .5 * len(a))
    return float(np.sum((a - e) * np.log(a / e)))


def compare(reference, current):
    result = {}
    for feature, expected in reference.items():
        actual = current[feature]
        ks = ks_2samp(finite_sample(expected), finite_sample(actual))
        result[feature] = dict(psi=psi(expected, actual),
                               ks=float(ks.statistic),
pvalue=float(ks.pvalue),
                               n_reference=len(expected),
```

```
n_current=len(actual))
    return result


def severity(features):
    # Bonferroni family threshold for three features, plus PSI effect-size
 gate.
    changed = [v for v in features.values() if v['pvalue'] < .01 / 3]
    if sum(v['psi'] >= .25 for v in changed) >= 2:
        return 'critical'
    if sum(v['psi'] >= .1 for v in changed) >= 2:
        return 'warning'
    return None


class AlertManager:
    def __init__(self, cooldown=900):
        self.cooldown = cooldown
        self.last = {}

    def check(self, features, now, defect=False, llm_failed=False):
        # LLM availability and contract defects must not wait for two
features.
        kind = ('llm_failure' if llm_failed else
                'data_defect' if defect else 'data_drift')
        level = 'critical' if (defect or llm_failed) else
severity(features)
        if level is None:
            return None
        previous = self.last.get(kind)
        escalation = previous and previous['level'] == 'warning' and level
 == 'critical'
        if previous and now - previous['at'] < self.cooldown and not
escalation:
            return dict(kind=kind, level=level, suppressed=True, at=now)
        self.last[kind] = dict(at=now, level=level)
        return dict(kind=kind, level=level, suppressed=False, at=now)
```

Додаток Б Журнали та метрики API

Лістинг програми telemetry.py. Оригінальний виконуваний файл включено до ZIP. Успадкований Engine ЛР12 повторно не друкується; тести та службові скрипти не включено до лістингу звіту.

```
from datetime import datetime, timezone
import json
import logging
import os
import time
import uuid

from fastapi.responses import Response
from prometheus_client import (CollectorRegistry, Counter, Histogram,
                               generate_latest, CONTENT_TYPE_LATEST)

MODEL_VERSION = 'flan-t5-small@0fc9ddf78a1e988dac52e2dac162b0ede4fd74ab'
PROMPT_VERSION = 'lab12-extractive-v1'


def install(app):
    registry = CollectorRegistry()
    requests = Counter('lab14_requests', 'Completed HTTP requests',
                       ['route', 'status'], registry=registry)
    errors = Counter('lab14_errors', 'HTTP errors including validation',
                     ['route', 'status'], registry=registry)
    latency = Histogram('lab14_latency_seconds', 'HTTP processing time',
                        ['route'], buckets=(.01, .05, .1, .5, 1, 2, 5, 10,
 30, 60),
                        registry=registry)
    tokens = Counter('lab14_tokens', 'Successful generation tokens',
                     ['kind'], registry=registry)
    cost = Counter('lab14_api_cost_usd', 'External API charge only',
registry=registry)
    cost.inc(0)
    logger = logging.getLogger('lab14.requests')
    logger.setLevel(logging.INFO)
    logger.propagate = False
    if not logger.handlers:
        handler = logging.StreamHandler()
        handler.setFormatter(logging.Formatter('%(message)s'))
        logger.addHandler(handler)

    @app.middleware('http')
    async def audit(request, call_next):
        request.state.rid = uuid.uuid4().hex
        start = time.perf_counter()
        status = 500
```

```
        try:
            response = await call_next(request)
            status = response.status_code
            response.headers['X-Request-ID'] = request.state.rid
            return response
        finally:
            # Template route labels prevent unbounded cardinality from
source ids.
            route = getattr(request.scope.get('route'), 'path',
'unmatched')
            elapsed = time.perf_counter() - start
            if route not in ('/metrics', '/health'):
                requests.labels(route, str(status)).inc()
                latency.labels(route).observe(elapsed)
                if status >= 400:
                    errors.labels(route, str(status)).inc()
                inp = getattr(request.state, 'prompt_tokens', 0)
                out = getattr(request.state, 'output_tokens', 0)
                tokens.labels('input').inc(inp)
                tokens.labels('output').inc(out)
                row =
dict(timestamp=datetime.now(timezone.utc).isoformat(),
                             request_id=request.state.rid, route=route,
status=status,
                             model_version=MODEL_VERSION,
prompt_version=PROMPT_VERSION,
                             duration_ms=round(elapsed * 1000, 3),
                             prompt_tokens=inp, output_tokens=out,
tokens=inp + out,
                             token_accounting='successful_generations_only',
                             api_cost_usd=0.0)
                logger.info(json.dumps(row, ensure_ascii=False))
                path = os.getenv('REQUEST_LOG')
                if path:
                    with open(path, 'a', encoding='utf-8') as stream:
                        stream.write(json.dumps(row, ensure_ascii=False) +
 '\n')

    @app.get('/metrics', include_in_schema=True)
    def metrics():
        return Response(generate_latest(registry),
media_type=CONTENT_TYPE_LATEST)

    app.state.metrics_registry = registry
```

Додаток В Відтворюваний експеримент дрейфу

Лістинг програми monitor.py. Оригінальний виконуваний файл включено до ZIP. Успадкований Engine ЛР12 повторно не друкується; тести та службові скрипти не включено до лістингу звіту.

```
from pathlib import Path
import hashlib
import json
import platform

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import numpy as np
import scipy

from drift import compare, severity, AlertManager

R = Path(__file__).resolve().parent
FEATURES = {
    'activePowerKw': ('Lab8Observation', 'timestamp', 'Active power, kW',
0, 20),
    'voltageV': ('Lab8Observation', 'timestamp', 'Voltage, V', 180, 270),
    'forecastKwh': ('Lab8Forecast', 'targetTime', 'Forecast energy, kWh',
0, 20),
}


def contract_errors(batch):
    errors = []
    for feature, (_, _, _, lo, hi) in FEATURES.items():
        values = np.asarray(batch.get(feature, []), dtype=float)
        if len(values) < 1000 or not np.isfinite(values).all():
            errors.append(feature + ': missing/nonfinite/insufficient
values')
        elif np.any((values < lo) | (values > hi)):
            errors.append(feature + ': outside agreed physical/unit
range')
    return errors


def json_write(name, value):
    (R / name).write_text(json.dumps(value, ensure_ascii=False, indent=2),
 'utf-8')


def main():
    (R / 'results').mkdir(exist_ok=True)
```

```
    (R / 'figures').mkdir(exist_ok=True)
    rng = np.random.default_rng(42)
    path = R / 'data/nodes.json'
    nodes = json.loads(path.read_text('utf-8'))
    reference, pool, info = {}, {}, {}
    for feature, (label, timestamp, title, lo, hi) in FEATURES.items():
        rows = sorted((n['properties'] for n in nodes if label in
n['labels']),
                      key=lambda p: p[timestamp])
        # Independent splits from a historical pool; not a live
time-series claim.
        rows = rows[:6000]
        assert len(rows) == 6000
        order = rng.permutation(len(rows))
        ref_ids, cur_ids = order[:3000], order[3000:]
        values = np.array([p[feature] for p in rows])
        reference[feature], pool[feature] = values[ref_ids],
values[cur_ids]
        info[feature] = dict(label=label, unit=title.split(', ')[-1],
                             n_reference=3000, n_pool=3000,
                             historical_start=rows[0][timestamp],
                             historical_end=rows[-1][timestamp],
                             mean=float(reference[feature].mean()),
                             std=float(reference[feature].std(ddof=1)),
                             reference_ids=[rows[int(i)][timestamp] for i
in ref_ids],
                             current_ids=[rows[int(i)][timestamp] for i in
 cur_ids])
    json_write('data/reference.json', {k: v.tolist() for k, v in
reference.items()})
    json_write('data/current_pool.json', {k: v.tolist() for k, v in
pool.items()})
    json_write('data/reference_manifest.json', dict(
        seed=42,
source_sha256=hashlib.sha256(path.read_bytes()).hexdigest(),
        design='first 6000 historical rows per population; seeded disjoint
 random split 3000/3000',
        scope='graph input features and stored forecast scores; not RAG
request score drift',
        features=info))
    cases = {
        'calm': pool,
        'mean_plus_1sigma': {k: v + reference[k].std(ddof=1) for k, v in
pool.items()},
        'units_times_100': {k: v * 100 for k, v in pool.items()},
    }
    results = {}
    for name, values in cases.items():
        metrics = compare(reference, values)
        errors = contract_errors(values)
```

```
        alert = AlertManager().check(metrics, 0, defect=bool(errors))
        results[name] = dict(metrics=metrics, contract_errors=errors,
alert=alert)
        fig, axes = plt.subplots(1, 3, figsize=(12, 3.6),
layout='constrained')
        for ax, (feature, (_, _, title, _, _)) in zip(axes,
FEATURES.items()):
            combined = np.r_[reference[feature], values[feature]]
            bins = np.linspace(combined.min(), combined.max(), 45)
            ax.hist(reference[feature], bins=bins, density=True,
histtype='step',
                    linewidth=1.8, label='Reference, n=3000')
            ax.hist(values[feature], bins=bins, density=True,
histtype='step',
                    linewidth=1.8, label='Current, n=3000')
            ax.set(xlabel=title, ylabel='Density',
title=f"PSI={metrics[feature]['psi']:.3f}")
            ax.legend(fontsize=8)
        fig.savefig(R / f'figures/{name}.png', dpi=180)
        plt.close(fig)
    json_write('results/drift.json', results)

    # Weeks are deliberately modeled windows, not calendar observations.
    weekly = []
    for week, shift in enumerate([0, 0, .1, .2, .35, .5, .75, 1, 1, 0],
1):
        batch = {k: rng.choice(v, 1000) + shift * reference[k].std(ddof=1)
                 for k, v in pool.items()}
        metrics = compare(reference, batch)
        weekly.append(dict(simulated_week=week, sigma_shift=shift,
metrics=metrics,
                           level=severity(metrics)))
    json_write('results/weeks.json', weekly)
    fig, ax = plt.subplots(figsize=(9, 4), layout='constrained')
    for feature in FEATURES:
        ax.plot([w['simulated_week'] for w in weekly],
                [w['metrics'][feature]['psi'] for w in weekly],
marker='o', label=feature)
    ax.axhline(.1, color='gray', linestyle='--', label='Warning 0.10')
    ax.axhline(.25, color='black', linestyle=':', label='Critical 0.25')
    ax.set(xlabel='Simulated week', ylabel='PSI', title='Modeled drift
over 10 windows')
    ax.legend()
    fig.savefig(R / 'figures/psi_weeks.png', dpi=180)
    plt.close(fig)

    quiet = []
    for i in range(100):
        batch = {k: rng.choice(v, 1000) for k, v in pool.items()}
        metrics = compare(reference, batch)
```

```
        quiet.append(dict(window=i, level=severity(metrics),
metrics=metrics))
    count = sum(w['level'] is not None for w in quiet)
    json_write('results/false_alarms.json', dict(
        n_windows=100, n_per_feature=1000, alerts=count, rate=count / 100,
        design='bootstrap of held-out historical pool; windows overlap,
not independent live weeks',
        windows=quiet))

    manager = AlertManager(cooldown=900)
    drift_metrics = results['mean_plus_1sigma']['metrics']
    alerts = [manager.check(drift_metrics, 0),
manager.check(drift_metrics, 60),
              manager.check(drift_metrics, 901),
              manager.check(results['units_times_100']['metrics'], 902,
defect=True),
              manager.check({}, 903, llm_failed=True)]
    # An availability failure is injected in a separate API test, not
inferred from PSI.
    json_write('results/alerts.json', dict(cooldown_seconds=900,
events=alerts,
                                           llm_scenario='modeled failure;
see fault_injection.json'))
    json_write('results/monitor_environment.json', dict(
        python=platform.python_version(), numpy=np.__version__,
scipy=scipy.__version__,
        matplotlib=matplotlib.__version__, seed=42))
    print(json.dumps(dict(drift_levels={k: v['alert'] for k, v in
results.items()},
                          quiet_alerts=count), ensure_ascii=False))


if __name__ == '__main__':
    main()
```

Додаток Г Прогін HTTP-запитів

Лістинг програми load_test.py. Оригінальний виконуваний файл включено до ZIP. Успадкований Engine ЛР12 повторно не друкується; тести та службові скрипти не включено до лістингу звіту.

```
from pathlib import Path
from datetime import datetime, timezone
import json
import time
import urllib.request
import urllib.error

import numpy as np

R = Path(__file__).resolve().parent
BASE = 'http://127.0.0.1:8144'


def fetch(route, payload=None):
    data = json.dumps(payload).encode() if payload is not None else None
    req = urllib.request.Request(BASE + route, data=data,
                                 headers={'Content-Type':
'application/json'})
    try:
        with urllib.request.urlopen(req, timeout=90) as response:
            return response.status, response.read().decode()
    except urllib.error.HTTPError as error:
        return error.code, error.read().decode()


def main():
    start = datetime.now(timezone.utc).isoformat()
    before = fetch('/metrics')[1]
    (R / 'results/metrics_before.prom').write_text(before, 'utf-8')
    questions = json.loads((R / 'data/questions.json').read_text('utf-8'))
    rows = []
    for i in range(50):
        # 45 real generations, five intentional invalid requests, serial
CPU load.
        if i % 10 == 9:
            payload = dict(question='x', top_k=8)
            expected = 422
        else:
            payload = dict(question=questions[i %
len(questions)]['question'],
                           top_k=3, use_graph=True)
            expected = 200
        t = time.perf_counter()
```

```
        status, text = fetch('/ask', payload)
        row = dict(sequence=i + 1, status=status,
expected_status=expected,
                   seconds=time.perf_counter() - t,
response=json.loads(text))
        rows.append(row)
        (R / 'results/load_requests.json').write_text(json.dumps(rows,
ensure_ascii=False, indent=2), 'utf-8')
        print(f'{i + 1}/50 HTTP {status}', flush=True)
    after = fetch('/metrics')[1]
    (R / 'results/metrics_after.prom').write_text(after, 'utf-8')
    times = np.array([r['seconds'] for r in rows])
    summary = dict(started_utc=start,
finished_utc=datetime.now(timezone.utc).isoformat(),
                   total=50, successful=sum(r['status'] == 200 for r in
rows),
                   intentional_validation_errors=sum(r['status'] == 422
for r in rows),
                   unexpected=sum(r['status'] != r['expected_status'] for
r in rows),
                   mean_seconds=float(times.mean()),
p50_seconds=float(np.quantile(times, .5)),
                   p95_seconds=float(np.quantile(times, .95)),
max_seconds=float(times.max()),
                   input_tokens=sum(r['response'].get('prompt_tokens', 0)
for r in rows),
                   output_tokens=sum(r['response'].get('output_tokens', 0)
 for r in rows),
                   api_cost_usd=0.0,
                   cost_scope='local model; excludes electricity, hardware
 and infrastructure',
                   success_p95_seconds=float(np.quantile([r['seconds'] for
 r in rows if r['status']==200], .95)),
                   first_request_seconds=rows[0]['seconds'],
                   design='45 real inference calls, five intentional
HTTP422; models loaded, first inference not warmed separately')
    (R / 'results/load_summary.json').write_text(json.dumps(summary,
indent=2), 'utf-8')
    print(json.dumps(summary, indent=2))
    if summary['unexpected']:
        raise SystemExit('Unexpected responses: inspect results before
accepting benchmark')


if __name__ == '__main__':
    main()
```